

Sprint 2 Manual: Catalog Data Foundation

Course: E-Commerce

Department of Computer Science / Artificial Intelligence

Institute of Mathematics & Computer Science, University of Sindh, Jamshoro

Sprint theme: Turn the Sprint 1 architecture and Week 3 catalog model into a reliable database foundation.

1 Purpose

Sprint 1 established the business boundary, MVP scope, technology choices, and initial ERD. Weeks 1–3 then added the concepts needed to represent a real catalog: products, categories, variants, SKUs, prices, stock, dynamic specifications, and image assets.

Sprint 2 is the first implementation sprint and lasts 15 days. Learners will build the catalog foundation that Sprint 3 and the later storefront, cart, and checkout can consume. The outcome is a coherent data model, migrations, basic administration, representative data, and automated verification.

2 Sprint goal and boundary

Given a product catalog administrator, the system must persist categories, products, variants, and SKUs without losing identity, relationship, price, or inventory meaning.

Duration and learner workload

15 calendar days. Teams should work in small, reviewable increments and demonstrate progress at the end of Days 5, 10, and 15.

In scope

- Category tree management with stable identifiers and slugs.
- Product creation and editing, including status and descriptive content.
- Variants and SKU records with unique codes, price, stock, and availability.
- Basic authenticated administration for categories, products, variants, and SKUs.
- Database constraints, migrations, seed data, and focused tests.

Out of scope

Dynamic specifications, asset upload, public catalog search, publication workflows, payment gateway integration, order placement, shipping integration, and the complete shopper checkout flow belong to Sprint 3 or later. Teams may stub these dependencies, but must not claim them as Sprint 2 functionality.

3 Required repository submission

Commit the following to the team's existing Sprint 1 repository:

1. `docs/SPRINT_2.md`, containing the design decisions and evidence requested below.
2. Database migrations or schema definitions for categories, products, variants, and SKUs.
3. Backend implementation for authenticated administrative create, read, update, and delete operations.
4. Seed or fixture data for at least three products, two categories, and four SKUs.
5. Automated model, validation, and authorization tests with the command used to run them.
6. A short README section explaining local setup and environment variables. Do not commit secrets.

4 Functional requirements

The following requirements are mandatory. Teams may add features only after these are complete.

ID	Capability	Acceptance requirement
CAT-01	Categories	An administrator can create, update, deactivate, and list categories. A category has a unique slug, an optional parent, and cannot become its own ancestor.
CAT-02	Product identity	An administrator can create and edit a product with name, slug, description, status, and category assignment. Slugs are unique within the catalog.
CAT-03	Variants and SKUs	A product can have zero or more variants and one or more sellable SKUs. Each SKU has a unique code and its own price and stock quantity.
CAT-04	Variant combinations	The system represents valid combinations only. A missing combination is not created as a fake or zero-stock SKU.
CAT-05	Data integrity	The database enforces required uniqueness and foreign-key constraints. API validation alone is not sufficient.
CAT-06	Administrative access	Administrative write operations reject unauthenticated or unauthorized requests.

5 Required data model

Extend the Sprint 1 ERD rather than replacing it without explanation. The Sprint 2 model must show the relationship between the original MVP entities and the catalog entities.

At minimum, include the following entities or their clearly justified equivalents:

- **Category:** id, parent id, name, slug, active status, timestamps.
- **Product:** id, category relationship, name, slug, description, status, timestamps.
- **Variant:** id, product id, option values or a reference to them.
- **SKU:** id, variant id, unique SKU code, price, stock quantity, active status.
- **Asset:** id, product or variant relationship, storage key or URL, role, alt text, sort order.
- **Specification:** either validated JSONB on Product or the EAV tables selected in Sprint 1, with a written validation rule.

Every relationship must identify its cardinality. Every foreign key must have an explicit delete/update policy. Price must use a suitable decimal or integer-minor-unit representation; floating-point money is not acceptable. Stock quantity must not become negative through a normal administrative update.

Required diagram

Add a Mermaid ER diagram to `docs/SPRINT_2.md`. It must include PK, FK, data types, cardinality, and the planned connection to `Carts`, `Cart_Items`, `Orders`, and `Order_Items` from Sprint 1.

```
erDiagram
    CATEGORIES ||--o{ PRODUCTS : contains
    PRODUCTS ||--o{ VARIANTS : has
    VARIANTS ||--o{ SKUS : materializes
    PRODUCTS ||--o{ ASSETS : displays
    PRODUCTS ||--o{ CART_ITEMS : selected_as
    SKUS ||--o{ ORDER_ITEMS : sold_as
```

6 Minimum administration contract

Document the actual routes implemented by the team. The following baseline is required unless the chosen framework uses an equivalent convention.

Method	Route	Purpose
POST	/api/v1/admin/products	Create a draft product; authenticated administrator only.
PATCH	/api/v1/admin/products/:id	Update product content or status; authenticated administrator only.
POST	/api/v1/admin/products/:id/skus	Create a new validated SKU; authenticated administrator only.
PATCH	/api/v1/admin/skus/:id	Update price, stock, or active status; authenticated administrator only.
GET	/api/v1/admin/products	Return administrative product records; authenticated administrator only.
POST	/api/v1/admin/categories	Create a category; authenticated administrator only.
GET	/api/v1/admin/categories	Return the category tree; authenticated administrator only.

Each endpoint must document request fields, response shape, status codes, validation errors, authentication requirements, and at least one example. Use consistent error responses. A request that attempts to create a duplicate slug or SKU must return a clear client error rather than a server traceback.

7 15-day work plan

Days	Focus	Checkpoint
1–3	Repository setup and migrations	Stack runs locally; migration creates the core tables.
4–6	Categories and products	Category tree and product CRUD work with constraints.
7–9	Variants and SKUs	SKU uniqueness, price, stock, and valid combinations are enforced.
10–12	Administration and seed data	Protected admin routes and reproducible sample data work.
13–14	Tests and documentation	Model, validation, authorization, ERD, and setup evidence are complete.
15	Sprint review	Demo, peer review, backlog refinement, and Sprint 3 hand-off.

8 Business rules and edge cases

The design document must answer these questions with examples from the implementation:

1. Can a draft product have no SKU? Can a published product have no sellable SKU?
2. Is a product assigned to one canonical category, many categories, or both? Why?
3. What happens when a parent category is deactivated?

4. How is an out-of-stock SKU represented in a public response?
5. Can two SKUs share a price? Can a SKU have a price override?
6. What prevents negative stock and duplicate SKU codes?
7. What happens to a product referenced by a future cart or order after it is deactivated?

9 Seed data and demonstration

The repository must include reproducible seed data that demonstrates the model, not merely one flat product row. Include:

- At least two levels in the category tree.
- At least three products, including one with multiple variants.
- At least four valid SKUs and one intentionally unavailable combination.
- At least one product with multiple variants and one unavailable SKU combination.

The demonstration must show an administrator creating a category, product, variant, and SKU, then retrieving the records through the administration API. Include request/response evidence in `docs/SPRINT_2.md`; redact tokens and private URLs.

10 Testing requirements

Automated tests must cover both successful behavior and rejection paths:

- Product and SKU creation with required fields.
- Duplicate slug and duplicate SKU rejection.
- Category hierarchy validation, including cycle prevention.
- Variant/SKU combination and stock rules.
- Authorization failure for administrative endpoints.

Record the test command and result. A manual screenshot is useful evidence, but it does not replace an automated test for a business rule.

11 Documentation format for `docs/SPRINT_2.md`

Use this order so another team member can review the work quickly:

1. Sprint goal and scope boundary.
2. Link to Sprint 1 decisions that this sprint reuses or changes.
3. Updated ERD and data dictionary.
4. Administration route table with examples.

5. Data integrity and authorization decisions.
6. Seed-data and demonstration instructions.
7. Test strategy, command, and result.
8. Known limitations and Sprint 3 backlog.

12 Evaluation rubric

Criterion	Weight	Evidence
Catalog data model and integrity	30%	Correct entities, constraints, cardinality, and migration/schema quality.
Administrative implementation	25%	Working protected CRUD operations and consistent validation.
Seed data and tests	20%	Reproducible data plus model, validation, and authorization tests.
Integration with Sprint 1	15%	Traceable reuse of scope, stack, ERD, and MVP assumptions.
Documentation and repository hygiene	10%	Clear Markdown, useful examples, clean commits, and no secrets.

13 Sprint review checklist

Before submission, each team should be able to answer “yes” to the following:

- Can a reviewer distinguish a product, variant, and SKU in the database and administration API?
- Are money, stock, slugs, and SKU codes protected by appropriate constraints?
- Can the seed command reproduce the same demonstration on a clean database?
- Do tests prove at least one failure path for every major business rule?
- Does the documentation explain what Sprint 3 can safely build on top of this foundation?

Sprint 3 hand-off

Sprint 3 must consume these catalog tables and SKU identities rather than duplicating product or pricing logic. Its backlog begins with dynamic specifications, assets, public catalog reads, publication rules, and catalog-to-cart readiness.